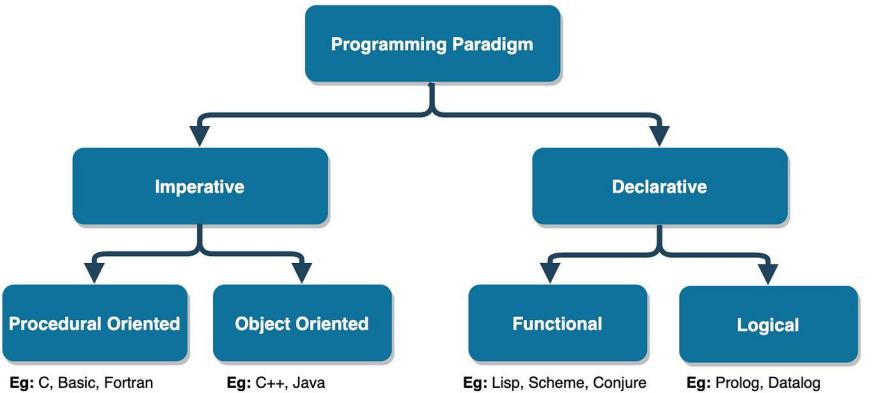
4.1 ООП


Programming Paradigm
Imperative
Declarative
Procedural Oriented
Object Oriented
Functional
Logical
Eg: C, Basic, Fortran
Eg: C++, Java
Eg: Lisp, Scheme, Conjure
Eg: Prolog, Datalog

Процедурное программирование — это парадигма программирования, которая основывается на концепции вызова процедур или функций.

Программы разделяются на процедуры, которые могут быть вызваны и выполнены в любом месте программы. Это позволяет разработчикам создавать более структурированные и понятные программы.

```c
#include <stdio.h>
#include <math.h>

typedef struct {
    double radius;
} Circle;

double calculate_area(Circle c) {
return M_PI * c.radius * c.radius;
}

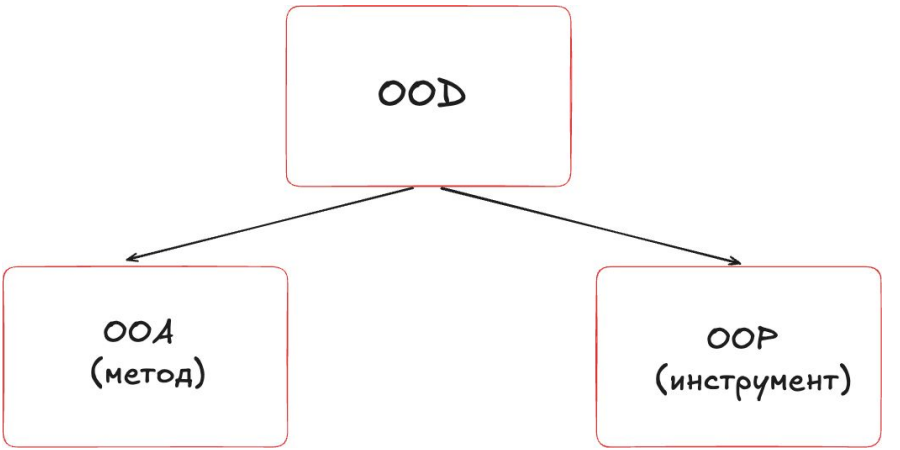
void print_area(double area) {
printf("Площадь круга: %f\n", area);
}

int main() {
   Circle my_circle;
   my_circle.radius = 5.0;
   double area = calculate_area(my_circle);
   print_area(area);

   return 0;
}
```

Объектно-ориентированное проектирование (OOD) — это подход к разработке программного обеспечения, который фокусируется на моделировании систем с использованием объектов

Объектный подход



ООА — это метод, который фокусируется на анализе требований и моделировании реальных объектов, их атрибутов и взаимодействий. Это начальный этап проектирования, где определяются сущности системы и их отношения.

Например, при разработке системы управления библиотекой ООА может включать:

- **Определение объектов**: "Книга", "Читатель", "Библиотекарь".
- **Атрибуты**: Для книги — название, автор; для читателя — имя, список выданных книг.
- **Взаимодействия**: Читатель может взять книгу, библиотекарь может добавить новую книгу.

ООР — это инструмент, который позволяет представить предметную область как набор объектов с данными и поведением (методами).

Структуры

Аналог классов, содержащие данные (поля)

```
type Circle struct {
   Radius float64
}
```

Методы

Функции, связанные со структурами, для реализации поведения.

```
type Circle struct {
  Radius float64
}

func (c Circle) Area() float64 {
   return math.Pi * c.Radius * c.Radius
}
```

Интерфейсы

Для достижения полиморфизма и абстракции

```go
package main

import "fmt"

type Shape interface {
    Area() float64
}

type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

func PrintArea(s Shape) {
    fmt.Println("Площадь:", s.Area())
}

func main() {
    c := Circle{Radius: 5}
    PrintArea(c)
}
```

Плюсы и минусы

Преимущества OOD включают:

- **Модульность**: Легко добавлять новые типы, реализующие интерфейсы, без изменения существующего кода.
- **Масштабируемость**: Системы, спроектированные с использованием OOD, легче расширять.
- **Поддерживаемость**: Разделение анализа и реализации упрощает отладку и обновление.

Вызовы включают:

- **Сложность проектирования**: Требуется глубокое понимание домена для эффективного ООА.
- **Избыточная абстракция**: Важно не злоупотреблять абстракциями, чтобы избежать излишней сложности.

ООП в Go

Go можно сказать, Object-based, но не Object-oriented язык, так как там нет прямого наследования и строгой иерархии классов.